

UNIVERSIDADE FEDERAL DO RIO GRANDE — FURG

Centro de Ciências Computacionais — C3

Sistemas para Internet 2 — Turma U — 1º Semestre de 2026

Nomes: Luis Henrique K. Reichow - 162589 e Matheus N. Buttow - 164400

RELATÓRIO TÉCNICO

Web Proxy com Controle de Conteúdo

Prof. Dr. André Prisco Vargas

Rio Grande, 2026

1. Justificativa da Tecnologia Escolhida

Para a implementação do proxy HTTP, a dupla optou pela linguagem Python com o microframework Flask. Essa escolha se justifica por múltiplos fatores técnicos e práticos.

Em primeiro lugar, o Flask oferece uma abstração de alto nível sobre o protocolo HTTP, permitindo que a lógica de roteamento e processamento de requisições seja implementada de forma clara e concisa. Isso reduziu o tempo de desenvolvimento e facilitou a leitura e manutenção do código.

Em segundo lugar, a biblioteca requests do Python simplifica o envio de requisições ao servidor de origem, oferecendo suporte nativo a todos os métodos HTTP (GET, POST, PUT, DELETE, PATCH) e personalização de cabeçalhos.

Comparado à alternativa de sockets TCP puros, o Flask elimina a necessidade de implementar manualmente o parsing do protocolo HTTP, tarefa complexa e propensa a erros. Em relação a tecnologias como Node.js ou Spring, a escolha pelo Python reflete a familiaridade da dupla com o ecossistema e o suporte técnico garantido pelo professor para essa stack.

1.1 Vantagens Percebidas

- Código limpo e legível, com roteamento declarativo via decoradores Flask
- Biblioteca requests robusta para chamadas HTTP ao servidor de origem
- Facilidade na manipulação de strings e expressões regulares para o filtro de conteúdo
- Excelente suporte a JSON nativo (módulo json da biblioteca padrão)

1.2 Dificuldades Encontradas

- Gerenciamento de cabeçalhos: cabeçalhos como Content-Length, Content-Encoding e Accept-Encoding exigiram tratamento específico. O Content-Length precisa ser removido para o requests recalcular automaticamente, e o Accept-Encoding precisa ser removido para forçar o servidor a devolver conteúdo sem compressão, permitindo que o proxy inspecione e modifique o texto
- Encoding de páginas: alguns sites declaravam ISO-8859-1 mas enviavam conteúdo em UTF-8, causando caracteres corrompidos. Foi necessário usar `apparent_encoding` da biblioteca requests para detectar o encoding correto antes de processar o HTML
- URLs relativas: recursos como imagens, arquivos CSS e scripts com caminhos relativos precisaram de lógica extra com `urljoin` e o cabeçalho Referer para ser reconstruídos corretamente e redirecionados pelo proxy
- Barra dupla do Flask: o Flask remove automaticamente barras duplas das rotas, corrompendo URLs no formato `http://destino.com`. Foi necessário detectar e corrigir esse comportamento manualmente

2. Decisões de Implementação

2.1 Rota Genérica com path:url

A rota principal do proxy captura qualquer caminho via o padrão `/<path:url>`, permitindo que URLs completas (`http://...`) e caminhos relativos sejam interceptados pela mesma função. O cliente acessa o proxy no formato `http://localhost:5000/http://destino.com`, e o Flask captura tudo após a primeira barra como o parâmetro `url`.

2.2 Modo de Uso: Navegador e cURL

O proxy foi implementado para receber a URL de destino diretamente no caminho da requisição, no formato esperado pelo enunciado: `http://localhost:5000/http://destino.com`. Nesse modo, o Flask identifica o host como `localhost:5000` e extrai a URL de destino do próprio caminho.

O cURL é utilizado sem a flag `-x`, passando a URL diretamente no caminho:

- `curl -v http://127.0.0.1:5000/http://httpbin.org/get`
- `curl -v -X POST -d 'usuario=luis' http://127.0.0.1:5000/http://httpbin.org/post`
- `curl -v http://127.0.0.1:5000/http://www.sitex.com` (retorna 403, bloqueado)

2.3 Limpeza de Cabeçalhos

Três cabeçalhos são removidos antes de repassar a requisição ao servidor de origem. O Host é removido para que o servidor alvo não receba `localhost:5000` como destino. O Content-Length é removido porque a biblioteca `requests` o recalcula automaticamente, evitando conflitos. O Accept-Encoding é removido para que o servidor retorne o conteúdo sem compressão, permitindo que o proxy leia e modifique o texto normalmente.

2.4 Filtro Case-Insensitive com Proteção de Tags HTML

A substituição de palavras utiliza expressões regulares com a flag `re.IGNORECASE`, garantindo que variações como 'Merda', 'MERDA' e 'merda' sejam todas substituídas. Além disso, o padrão `regex` inclui um lookahead negativo (`?![<]*>`) que impede substituições dentro de atributos de tags HTML, evitando corromper URLs, classes CSS ou outros atributos que possam conter as palavras filtradas.

O mapa de substituições é carregado uma vez na inicialização do servidor a partir do arquivo `words.json`, evitando leituras desnecessárias de disco a cada requisição.

2.5 Reescrita de Links e Recursos

Para manter a navegação funcional pelo proxy, o conteúdo HTML retornado tem seus links reescritos antes de ser entregue ao cliente. A função `rewrite_links` percorre atributos `href`, `src` e `action` e prefixa URLs absolutas com o endereço do proxy, garantindo que cliques em links e carregamento de recursos continuem passando pelo proxy.

Arquivos CSS também são processados: URLs relativas declaradas via `url()` são reescritas para passar pelo proxy, permitindo que imagens de fundo e fontes sejam carregadas corretamente.

2.6 Log de Acessos

Cada requisição é registrada em `log.txt` com timestamp, URL e ação executada (permitido, filtrado ou bloqueado). Para determinar se houve filtragem, o conteúdo original é comparado com o conteúdo processado antes de converter para bytes, garantindo que a comparação seja feita entre strings e não entre tipos diferentes.

2.7 Tratamento de Erros de Conexão

A função `send_req` é envolta em um bloco `try/except` que captura exceções da biblioteca `requests`. Caso o servidor de destino esteja inacessível ou a URL seja inválida, o proxy retorna uma resposta HTTP 502 com uma mensagem descritiva, sem expor detalhes internos da aplicação.

3. HTTPS e o Filtro de Conteúdo

3.1 Por que o filtro não funciona em sites HTTPS no modo proxy nativo?

O protocolo HTTPS adiciona uma camada de criptografia TLS/SSL entre o cliente e o servidor, cifrando todo o conteúdo da comunicação de ponta a ponta.

Quando um navegador está configurado para usar um proxy e tenta acessar um site HTTPS, ele envia ao proxy o método HTTP CONNECT, solicitando que o proxy estabeleça um túnel TCP direto até o servidor de destino. Nesse modo, o proxy apenas encaminha bytes cifrados sem conseguir inspecioná-los ou modificá-los, pois não possui as chaves criptográficas da sessão TLS.

Vale observar que, no modo de uso implementado, onde a URL é passada diretamente no caminho da requisição (`http://localhost:5000/https://destino.com`), a biblioteca requests consegue buscar sites HTTPS normalmente, pois ela própria estabelece a conexão TLS e decifra o conteúdo antes de entregá-lo ao proxy. Nesse caso, o filtro de palavras funciona normalmente. A limitação se aplica apenas ao modo proxy nativo via CONNECT.

3.2 O que seria necessário para funcionar no modo nativo?

Para inspecionar e modificar tráfego HTTPS no modo proxy nativo, seria necessário implementar um proxy SSL interceptor. Isso envolveria:

- Geração de uma Autoridade Certificadora (CA) própria, instalada nos clientes como confiável
- Ao receber uma conexão HTTPS via CONNECT, o proxy gera um certificado falso assinado pela CA local
- O proxy decifra o tráfego, aplica os filtros, e re-cifra antes de encaminhar ao servidor real
- Ferramentas como mitmproxy implementam exatamente essa abordagem

Essa solução tem implicações éticas e legais significativas, pois rompe o modelo de confiança do HTTPS. Em ambientes corporativos, é utilizada com consentimento explícito dos usuários e certificados instalados administrativamente.

4. O que Aprendemos sobre o Protocolo HTTP

- Estrutura real das requisições: compreendemos na prática o papel dos cabeçalhos como Host, Content-Type, Content-Length e Accept-Encoding, e como cada um afeta o comportamento de servidores e proxies.
- Modelo cliente-servidor intermediado: entendemos como um proxy atua simultaneamente como servidor para o cliente e como cliente para o servidor de origem, duplicando seu papel na comunicação.
- Diferença entre HTTP e HTTPS: a implementação tornou concreta a compreensão de por que o HTTPS impede a inspeção de conteúdo por terceiros no modo proxy nativo, e por que no modo de URL direta isso é possível através da biblioteca requests.
- Método CONNECT: aprendemos que navegadores usam esse método especial para solicitar ao proxy que estabeleça um túnel TCP direto para sites HTTPS.
- Redirecionamentos HTTP (301/302): percebemos que o comportamento padrão da biblioteca requests de seguir redirecionamentos automaticamente precisava ser desativado (`allow_redirects=False`) para que o cliente recebesse a resposta original e pudesse decidir como proceder.

- Encoding de conteúdo: descobrimos que servidores podem declarar um encoding incorreto nos cabeçalhos, e que a detecção automática via `apparent_encoding` é necessária para renderizar corretamente páginas com caracteres especiais.

5. Uso de Inteligência Artificial

A dupla utilizou o assistente Claude (Anthropic) como ferramenta de apoio ao longo do desenvolvimento.

5.1 Como foi utilizado

- Geração e revisão do README.md do repositório
- Esclarecimento de dúvidas sobre o comportamento da biblioteca Flask e requests
- Geração deste relatório técnico, com base no código desenvolvido pela dupla e no enunciado do trabalho
- Revisão de trechos de código para identificar possíveis bugs e sugestão de melhorias
- Testes estáticos do código: simulação de cenários de borda para validar funções como `blocked()`, `filter_word()` e `rewrite_links()` sem necessidade de execução real

5.2 O que foi modificado

Todo o código-fonte do proxy foi escrito e compreendido pela dupla. A IA foi utilizada principalmente para documentação, esclarecimento de conceitos e identificação de bugs, não para geração direta de código funcional. Os textos gerados foram revisados e adaptados para refletir com precisão as decisões técnicas efetivamente tomadas.

5.3 O que aprendemos com a interação

A interação com a IA foi útil para estruturar o raciocínio sobre problemas como o tratamento de HTTPS, a distinção entre os modos de uso do proxy e o comportamento de cabeçalhos HTTP. As respostas geradas serviram como ponto de partida para investigação, sendo sempre verificadas com testes práticos e documentação oficial.